



HEROback

Memoria Técnica del Proyecto

Plataforma de colección y gestión de superhéroes

Backend: Express + PostgreSQL + Sequelize

Frontend: React 18 + Vite 5

2026

Contenido

- 1. Introducción 3
- 2. Tecnologías utilizadas 3
 - 2.1 Backend 3
 - 2.2 Frontend..... 3
- 3. Arquitectura del sistema 4
 - 3.1 Capa de presentación (Frontend React)..... 4
 - 3.2 Capa de negocio (Backend Express) 4
 - 3.3 Capa de datos (PostgreSQL) 4
- 4. Descripción del backend 4
 - 4.1 API REST 4
 - 4.2 Proxy de imágenes..... 5
 - 4.3 Autenticación y autorización 5
 - 4.4 Seeding de datos 6
- 5. Descripción del frontend 6
 - 5.1 Componentes..... 6
 - Componentes reutilizables (src/components/) 6
 - Páginas (src/pages/)..... 6
 - 5.2 Hooks personalizados 7
 - 5.3 Gestión del estado 7
 - 5.4 Sistema de temas..... 7
- 6. Mejoras realizadas sobre el backend original..... 7
 - 6.1 Problema de imágenes (hotlink-protection) 7
 - 6.2 Gestión de héroes en equipos sin implementar..... 8
 - 6.3 Código muerto eliminado..... 8
- 7. Consideraciones de seguridad..... 8
- 8. Guía de despliegue..... 8
 - 8.1 Desarrollo local 8
 - 8.2 Producción 9
- 9. Estructura de ficheros del proyecto 9
 - 9.1 HEROback-front/ 9
- 10. Conclusiones 9

1. Introducción

HEROback es una plataforma web fullstack para la gestión y colección de superhéroes. Permite a los usuarios explorar un catálogo de personajes con sus estadísticas reales, añadirlos a su colección personal, formar equipos de combate y adquirir productos en una tienda interna con un sistema de moneda virtual (coins).

El proyecto está dividido en dos capas independientes que se comunican exclusivamente a través de una API REST:

- Backend: aplicación Express.js con base de datos PostgreSQL, gestionada mediante Sequelize ORM.
- Frontend: aplicación React 18 construida con Vite 5, que consume la API del backend.

2. Tecnologías utilizadas

2.1 Backend

Tecnología	Versión	Uso
Node.js	18+	Runtime JavaScript del servidor
Express.js	4.x	Framework HTTP y routing
Sequelize	6.x	ORM para PostgreSQL
PostgreSQL	14+	Base de datos relacional
JSON Web Token	9.x	Autenticación stateless
bcryptjs	2.x	Hash seguro de contraseñas
Pug	3.x	Motor de plantillas (vistas legacy)
Docker Compose	—	PostgreSQL + pgAdmin en contenedores

2.2 Frontend

Tecnología	Versión	Uso
React	18.3	Librería UI con componentes funcionales
Vite	5.4	Bundler + dev server con HMR
React Router DOM	6.26	Enrutamiento SPA del lado cliente
CSS Variables	nativo	Sistema de temas claro/oscuro
JSDoc	nativo	Documentación en código
localStorage API	nativo	Persistencia de sesión, tema y filtros

3. Arquitectura del sistema

El sistema sigue una arquitectura cliente-servidor desacoplada de tres capas:

3.1 Capa de presentación (Frontend React)

La interfaz de usuario se construye como una Single Page Application (SPA). Vite sirve los estáticos en el puerto 5173 y actúa como proxy inverso en desarrollo, redirigiendo todas las peticiones `/api/*` al backend en el puerto 3000. Esta estrategia elimina los problemas de CORS en desarrollo y permite que las URLs de imagen relativas (`/api/heroes/:id/image`) funcionen sin configuración adicional.

3.2 Capa de negocio (Backend Express)

El backend expone una API REST versionada bajo `/api/`. Cada dominio de negocio sigue el patrón Routes → Controller → Service → Model:

- Routes: define los endpoints y aplica middlewares (autenticación, roles, queryFeatures).
- Controller: valida la request, llama al servicio y formatea la response.
- Service: contiene la lógica de negocio y las operaciones de BD.
- Model: define el esquema Sequelize y las asociaciones.

3.3 Capa de datos (PostgreSQL)

La base de datos relacional gestiona siete entidades principales:

Tabla	Descripción
users	Usuarios del sistema (id, username, email, password_hash, role, coins)
heroes	Catálogo de superhéroes con stats (intelligence, strength, speed...)
products	Artículos disponibles en la tienda (name, price, stock)
user_heroes	Colección personal: qué héroes posee cada usuario
user_products	Historial de compras de productos por usuario
teams	Equipos de combate creados por cada usuario
team_heroes	Relación N:M entre equipos y los user_heroes que los componen

4. Descripción del backend

4.1 API REST

La API sigue convenciones REST. Todos los endpoints devuelven JSON. Los endpoints protegidos requieren el header `Authorization: Bearer <token>`. Los endpoints de lista soportan paginación, filtros y ordenamiento mediante query parameters procesados por el middleware `queryFeatures`.

Endpoint	Método	Descripción
/api/auth/register	POST	Registro de nuevos usuarios
/api/auth/login	POST	Login — devuelve JWT + perfil
/api/auth/me	GET	Perfil del usuario autenticado
/api/heroes	GET	Listado paginado con filtros
/api/heroes/:id	GET	Detalle de un héroe
/api/heroes/:id/image	GET	Proxy de imagen (anti-hotlink)
/api/heroes/mis-heroes	GET	Colección del usuario actual
/api/store/add-hero	POST	Añadir héroe a la colección
/api/store/buy	POST	Comprar un producto
/api/teams	GET/POST	Listar/crear equipos del usuario
/api/teams/:id	PATCH/DELETE	Actualizar/eliminar un equipo
/api/teams/:id/heroes	POST	Añadir héroe al equipo
/api/teams/:id/heroes/:uhId	DELETE	Quitar héroe del equipo
/api/admin/stats	GET	Estadísticas globales (solo admin)
/api/users	GET	Lista de usuarios (solo admin)

4.2 Proxy de imágenes

Un problema común en proyectos que consumen imágenes de terceros es la hotlink-protection: el servidor externo (superherodb.com) bloquea solicitudes cuya cabecera Referer apunte a un dominio distinto. La solución implementada consiste en un proxy interno:

- El frontend siempre solicita /api/heroes/:id/image.
- El backend descarga la imagen del origen externo sin enviar Referer.
- La imagen se cachea en memoria durante 24 horas (hasta 500 entradas, política LRU).
- Si la descarga falla, el endpoint devuelve el placeholder.svg local.
- Se implementan guardas anti-SSRF que bloquean IPs privadas y locales.

4.3 Autenticación y autorización

La autenticación se implementa mediante JSON Web Tokens (JWT). El middleware verifyToken extrae y valida el token del header Authorization. El middleware checkRole(role) verifica que el usuario tenga el rol requerido. Las vistas Pug legacy usan sesiones de Express independientemente; la API REST solo usa JWT.

4.4 Seeding de datos

El proyecto incluye tres scripts de datos:

- `npm run seed` — siembra héroes desde el fichero local `heroes.json` (sin conexión externa).
- `npm run seed:live` — obtiene datos en tiempo real de la SuperHero API (requiere `SUPERHERO_API_KEY` en `.env`).
- `npm run seed:images` — actualiza solo las URLs de imagen desde la SuperHero API, sin alterar el resto de datos.

5. Descripción del frontend

5.1 Componentes

El proyecto cuenta con 18 componentes funcionales React distribuidos en dos categorías:

Componentes reutilizables (`src/components/`)

Componente	Responsabilidad
Navbar	Barra de navegación responsiva con menú hamburguesa en móvil
ThemeToggle	Botón ☀/☾ que llama a <code>toggle()</code> del <code>ThemeContext</code>
HeroCard	Tarjeta de héroe con imagen proxy, tag de alignment y publisher
HeroGrid	Grid con estados: cargando (skeletons), error y vacío
Modal	Modal accesible: cierra con Escape o clic en backdrop
ProtectedRoute	Guard de ruta: redirige a <code>/login</code> si no hay sesión activa
Footer	Pie de página con enlace y créditos

Páginas (`src/pages/`)

Página	Funcionalidad
HomePage	Landing con 12 héroes destacados
HeroesPage	Catálogo con búsqueda debounced, filtros y paginación
HeroDetailPage	Ficha completa con barras de stats y botón de colección
LoginPage / RegisterPage	Formularios de autenticación con validación
DashboardPage	Resumen de coins, héroes y equipos del usuario
ProfilePage	Edición de username y email
TeamsPage	Crear/eliminar equipos, añadir/quitar héroes (máx. 6)
StorePage	Tienda de productos con descuento de coins en tiempo real
AdminPage	Stats globales y gestión de usuarios (solo admin)
NotFoundPage	Página 404 con enlace al inicio

5.2 Hooks personalizados

Hook	Descripción
useFetch(fn, deps)	Ejecuta una promesa y expone { data, error, loading, refresh }. Cancela actualizaciones si el componente se desmonta.
useLocalStorage(key, init)	Sincroniza estado con localStorage. Multi-tab safe (evento storage). Acepta función de actualización.
useDebounce(value, delay)	Retrasa la propagación de un valor. Evita peticiones por cada tecla en el buscador.
useAuth()	Accede al AuthContext: user, login, logout, register, updateUser.
useTheme()	Accede al ThemeContext: theme, toggle, setTheme.

5.3 Gestión del estado

El estado de la aplicación se organiza en tres niveles:

- Global (Context): AuthContext (sesión y usuario) y ThemeContext (tema visual).
- Local de página (useState + useFetch): datos de cada página, estados de formularios, feedback de operaciones.
- Persistido (localStorage): token JWT, cache de usuario, tema elegido y filtros del catálogo.

5.4 Sistema de temas

El sistema de temas dual se implementa íntegramente en CSS sin dependencias externas. Las variables CSS se definen en src/styles/theme.css bajo los selectores :root (oscuro) y [data-theme="light"]:

- Tema oscuro: fondo #0b0b14, superficie #1c1c38, primario indigo #6366f1, acento ámbar #f59e0b.
- Tema claro: fondo crema #faf9f7, superficie #ffffff, primario indigo más saturado #4f46e5, acento ámbar #d97706.

El ThemeContext aplica el atributo data-theme en <html> en cada cambio, activando automáticamente el bloque de variables correspondiente. El estado inicial se obtiene de localStorage o de prefers-color-scheme del sistema operativo.

6. Mejoras realizadas sobre el backend original

Durante el desarrollo del proyecto se identificaron y corrigieron los siguientes problemas en el backend:

6.1 Problema de imágenes (hotlink-protection)

Las imágenes de los héroes en la base de datos apuntaban a superherodb.com, que aplica hotlink-protection bloqueando peticiones con Referer de otro dominio. El navegador recibía 403 y mostraba siempre el placeholder. Solución: se implementó el proxy de imágenes descrito en la sección 4.2.

6.2 Gestión de héroes en equipos sin implementar

La UI de equipos solo permitía crear y eliminar equipos. Los endpoints de backend para gestión granular (añadir/quitar héroes) no existían. Se añadieron:

- POST /api/teams/:id/heroes — añade un héroe (user_hero_id) al equipo, validando propiedad, duplicados y límite de 6.
- DELETE /api/teams/:id/heroes/:userHeroId — elimina un héroe del equipo.
- teamService.addHero y removeHero con asignación automática de posición libre.
- La vista teams.pug (backend) y TeamsPage.jsx (frontend) se rehacen con los nuevos controles.

6.3 Código muerto eliminado

Se eliminaron src/seeder/seedDatabase.js y resetAndSeed.js, que importaban los modelos Power y Comment inexistentes en models/index.js y habrían provocado un crash al ejecutarse.

7. Consideraciones de seguridad

- Contraseñas almacenadas con bcrypt (factor de coste 10).
- JWT con expiración configurable vía JWT_SECRET.
- El proxy de imágenes incluye guardas anti-SSRF: bloquea localhost, 127.0.0.1, ::1, rangos RFC-1918 (10.x, 172.16-31.x, 192.168.x) y protocolos no-http.
- Las rutas de admin verifican doble: middleware verifyToken + checkRole("admin").
- El frontend nunca expone el token en la URL ni en logs de consola.
- Los inputs de búsqueda pasan por el middleware queryFeatures que sanitiza y limita los parámetros aceptados.

8. Guía de despliegue

8.1 Desarrollo local

Requisitos: Node 18+, Docker (para PostgreSQL).

```
# Backend
cd HEROback && docker-compose up -d && npm install && npm run seed && npm run dev

# Frontend (en otra terminal)
cd HEROback-front && npm install && npm run dev
```


8.2 Producción

El frontend se compila en estáticos que se sirven con cualquier servidor web. El proxy de Vite no existe en producción; debe configurarse en el servidor web o plataforma de hosting.

```
# Build del frontend
cd HEROback-front && npm run build
# Los estáticos quedan en /dist
```

Para Nginx, añadir un bloque de proxy:

```
location /api/ { proxy_pass http://localhost:3000; }
location / { try_files $uri $uri/ /index.html; }
```

9. Estructura de ficheros del proyecto

9.1 HEROback-front/

src/api/client.js · resources.js — Capa de comunicación.

src/components/ — 7 componentes reutilizables.

src/context/ — AuthContext, ThemeContext.

src/hooks/ — useFetch, useLocalStorage, useDebounce.

src/pages/ — 11 vistas enrutadas.

src/styles/ — global.css + theme.css.

10. Conclusiones

HEROback es un proyecto fullstack completo que demuestra la integración de un backend REST con Express/PostgreSQL y un frontend React moderno. Las principales aportaciones técnicas del proyecto son:

- Arquitectura desacoplada: backend y frontend son completamente independientes y comunicados únicamente por API REST y JWT.
- Proxy de imágenes: solución elegante al problema de hotlink-protection, con caché LRU y guardas de seguridad.
- Sistema de temas dual: implementado sin dependencias, respetando la preferencia del sistema operativo.
- Hooks personalizados reutilizables: useFetch, useLocalStorage y useDebounce con un patrón claro y composable.
- Persistencia selectiva en localStorage: token, usuario, tema y filtros persisten entre sesiones de forma independiente y controlada.
- Gestión completa de equipos: los endpoints granulares addHero/removeHero resuelven la funcionalidad que faltaba en el backend original.